



飞桨新人串讲

静态图执行逻辑与内存管理

陈锐彪 2021.11.01.

01. 静态图模型执行逻辑

02. 内存管理模块设计

03. 常用显存优化方法

– 先看一份静态图的示例代码

```
x = static.data(name='X', shape=[None, 1])  
hidden = static.nn.fc(x, size=5)  
loss = paddle.mean(hidden)  
paddle.optimizer.SGD().minimize(loss)
```

```
exe = static.Executor()  
exe.run(static.default_startup_program())
```

```
compiled_prog = static.CompiledProgram(  
    static.default_main_program())
```

```
x = numpy.random.random(size=(5, 1)).astype('float32')  
loss_data, = exe.run(compiled_prog,  
                      feed={"X": x},  
                      fetch_list=[loss.name])
```

1. 声明前向网络
2. 定义损失函数
3. 自动微分、参数优化
4. 构造执行器
5. 执行startup_program
6. 编译main_program
7. 执行编译后的main_program

startup_program和main_program存储了模型的描述信息，通过Executor执行

— 模型在Python端用Program描述

```
class Program():  
    def __init__(self):  
        self.desc = core.ProgramDesc()  
        self.blocks = [Block(self, 0)]
```

```
class Block():  
    def __init__(self, program, idx):  
        self.desc = program.desc.block(idx)  
        self.vars = collections.OrderedDict()  
        self.ops = list()
```

class Variable()

输入输出

class Operator()

Python端

```
class ProgramDesc {  
    proto::ProgramDesc desc_;  
    std::vector<std::unique_ptr<BlockDesc>>  
        blocks_;  
};
```

```
class BlockDesc {  
    proto::BlockDesc *desc_;  
    std::deque<std::unique_ptr<OpDesc>>>ops_;  
    std::unordered_map<std::string,  
        std::unique_ptr<VarDesc>> vars_;  
};
```

class VarDesc()

输入输出

class OpDesc()

C++端

pybind

→ 以成员变量的方式持有

- 模型在C++端用ProgramDesc描述

```
class ProgramDesc {  
    proto::ProgramDesc desc_;  
    std::vector<std::unique_ptr<BlockDesc>>  
        blocks_;  
};  
  
class BlockDesc {  
    proto::BlockDesc *desc_;  
    std::deque<std::unique_ptr<OpDesc>> ops_;  
    std::unordered_map<std::string,  
        std::unique_ptr<VarDesc>> vars_;  
};
```

class VarDesc()

输入输出

class OpDesc()

```
message ProgramDesc {  
    reserved 2, 3; // For backward compatibility.  
    repeated BlockDesc blocks = 1;  
    optional Version version = 4;  
    optional OpVersionMap op_version_map = 5;  
}
```

```
message BlockDesc {  
    required int32 idx = 1;  
    required int32 parent_idx = 2;  
    repeated VarDesc vars = 3;  
    repeated OpDesc ops = 4;  
    optional int32 forward_block_idx = 5  
        [ default = -1 ];  
}
```

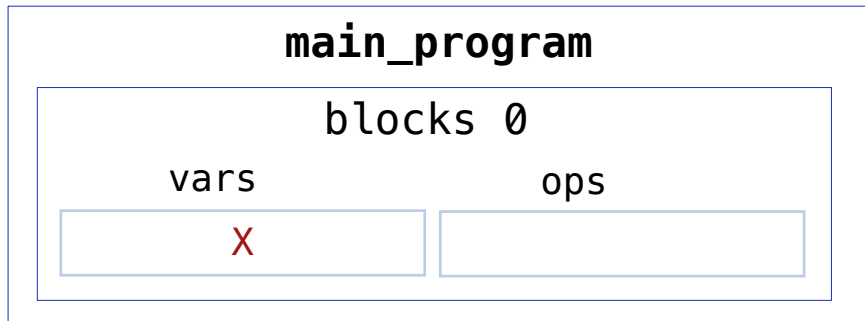
```
message VarDesc {  
    .....  
}
```

```
message OpDesc {  
    .....  
}
```

真正的模型描述信息以protobuf的形式保存

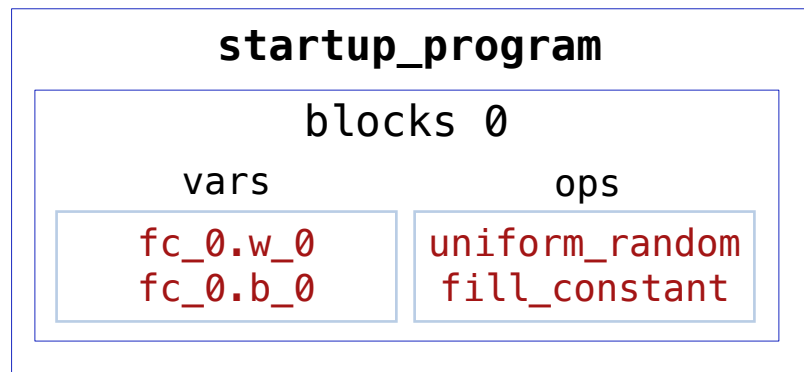
— 当用户声明了变量X

```
x = static.data(name='X', shape=[None, 1])
```

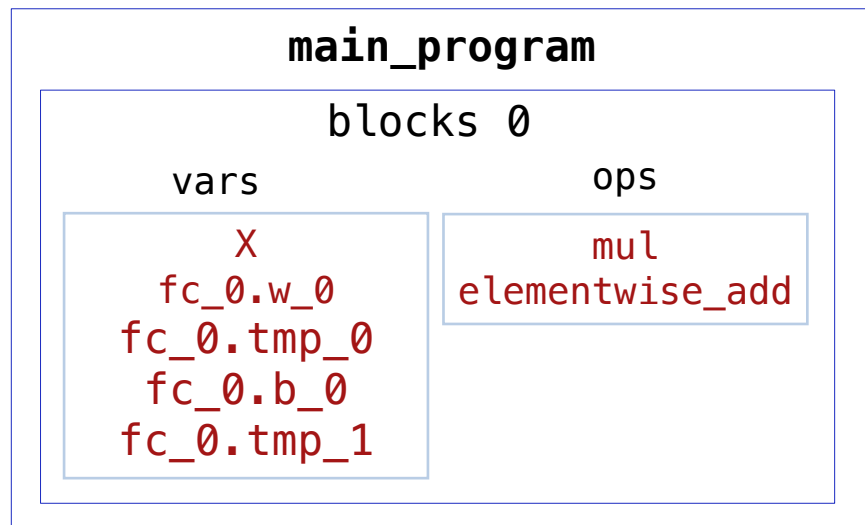


— 加上一个全连接层fc

`x = static.data(name='X', shape=[None, 1])`
`hidden = static.nn.fc(x, size=5)` $\longrightarrow$ $hidden = X * W + b$



```
fc_0.w_0 = uniform_random()  
fc_0.b_0 = fill_constant()
```



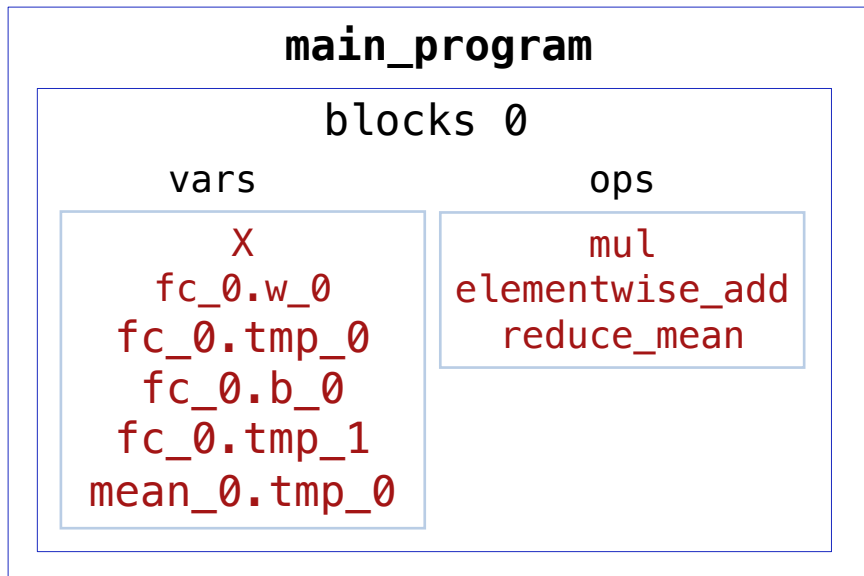
```
fc_0.tmp_0 = mul(X, fc_0.w_0)  
fc_0.tmp_1 = elementwise_add(fc_0.tmp_0, fc_0.b_0)
```

startup_program : 描述模型参数初始化、优化器参数初始化、reader初始化等操作

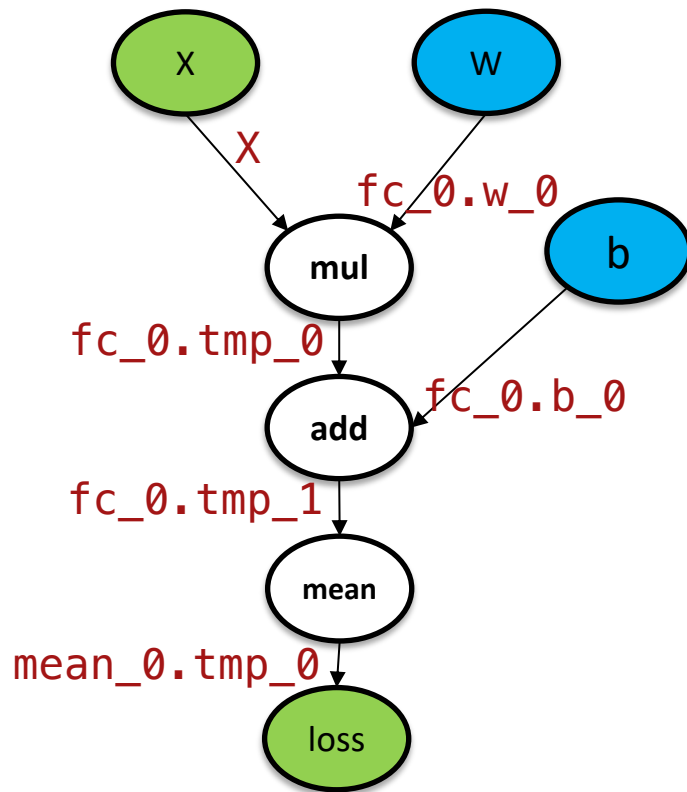
main_program : 描述模型前向反向操作、模型参数更新、优化器参数更新等操作

— 加上损失函数mean

```
x = static.data(name='X', shape=[None, 1])
hidden = static.nn.fc(x, size=5)
loss = paddle.mean(hidden)
```



$loss = mean(X * W + b)$



— 优化器的minimize做了什么？

```
paddle.optimizer.SGD().minimize(loss)
```

```
def minimize(self, loss, startup_program=None,  
             parameters=None, no_grad_set=None):  
    assert isinstance(loss, Variable), "The loss should be an Tensor."  
    parameter_list = parameters if parameters \  
        else self._parameter_list
```

```
    params_grads = self.backward(  
        loss,  
        startup_program=startup_program,  
        parameters=parameter_list,  
        no_grad_set=no_grad_set)
```

调用backward构反向图，
做自动微分

```
    optimize_ops = self._apply_optimize(  
        loss, startup_program=startup_program,  
        params_grads=params_grads)
```

调用_apply_optimize做参数优化

```
    return optimize_ops, params_grads
```

— backward自动微分的实现逻辑

核心代码实现在append_backward函数中，流程如下：

1. 调用_create_loss_op_desc构造一个fill_constant OP，将**loss的初始梯度赋值为1**
2. 调用_find_op_path**获取所有与loss计算相关的前向op**
3. 调用_find_no_grad_vars**获取所有与loss计算无关的变量**（不需要梯度）
4. 调用_append_backward_ops，按逆序为每个相关的前向OP**构造对应的反向OP**并添加到Block中，此步骤只会构造与loss梯度传递相关的反向OP，还实现了以下两个逻辑：
 - **梯度聚合**：实现在_addup_repetitive_outputs中，包含inplace addto优化策略
 - **静态图剪枝**：裁剪反向图中输入/输出变量不需要计算梯度的OP
5. 调用_append_backward_vars**创建每个反向OP输出的反向变量**，裁剪输入/输出中不存在反向变量的反向OP
6. 返回每个训练参数及其对应的梯度变量

– _apply_optimize 优化参数的实现逻辑

实际调用了apply_gradients，核心代码如下：

OP融合：

将所有参数/梯度的内存聚拢到一起，形成一个新tensor，后续的参数更新操作合成一次

```
def apply_gradients(self, params_grads):  
    params_grads = sorted(params_grads, key=lambda x: x[0].name)  
    if self._flatten_param_grads and self.regularization is None:  
        if self._grad_clip == None or isinstance(self._grad_clip,  
                                                  ClipGradByGlobalNorm):  
            params_grads = self.flatten_param_grads(params_grads)
```

```
# 'optimizer(grad_clip)' or 'set_gradient_clip'  
if self._grad_clip is not None:  
    params_grads = self._grad_clip(params_grads)  
else:  
    params_grads = append_gradient_clip_ops(params_grads)
```

梯度裁剪

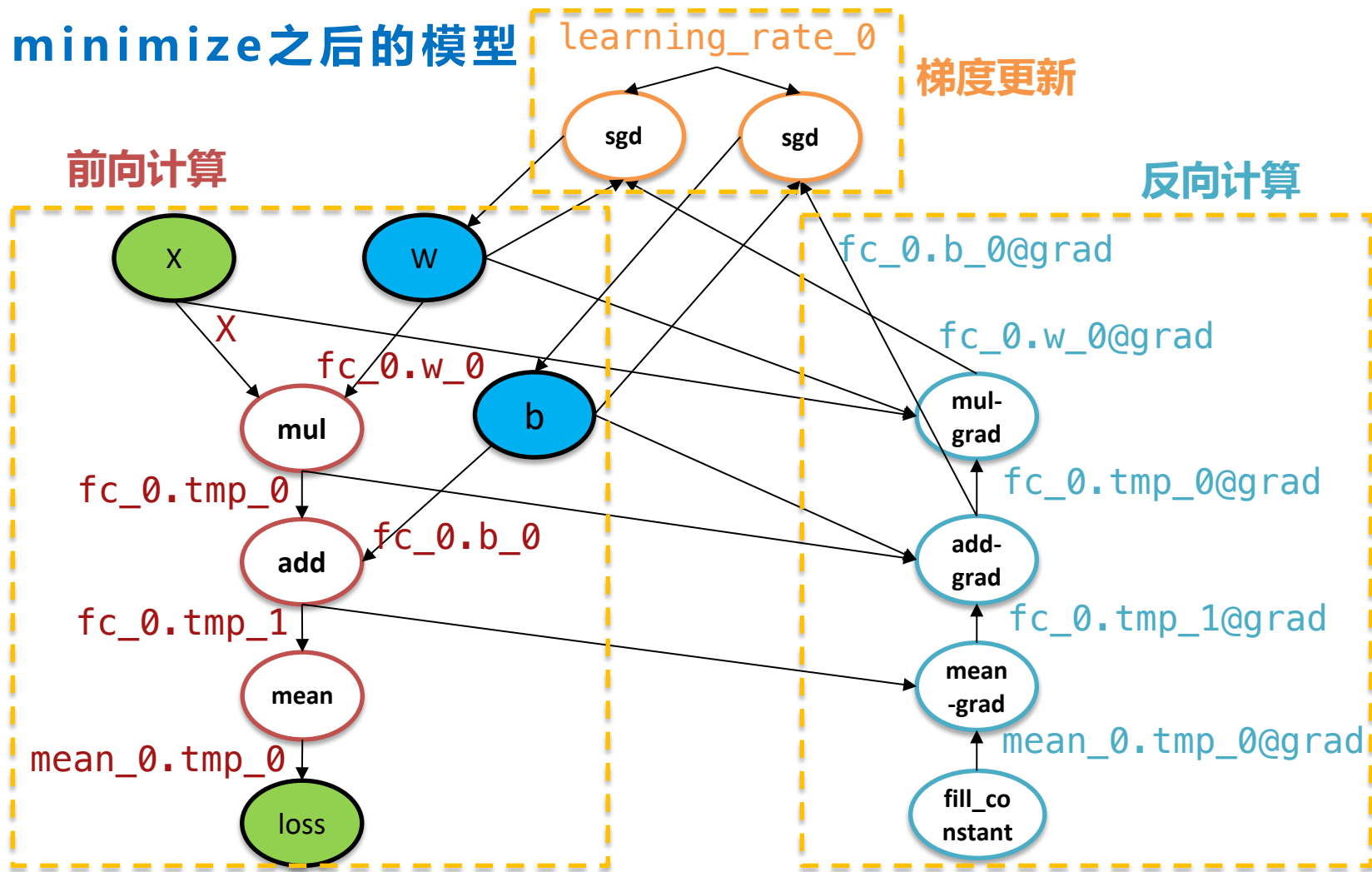
```
# Add regularization if any  
params_grads = self.append_regularization_ops(params_grads,  
                                              self.regularization)
```

梯度正则化

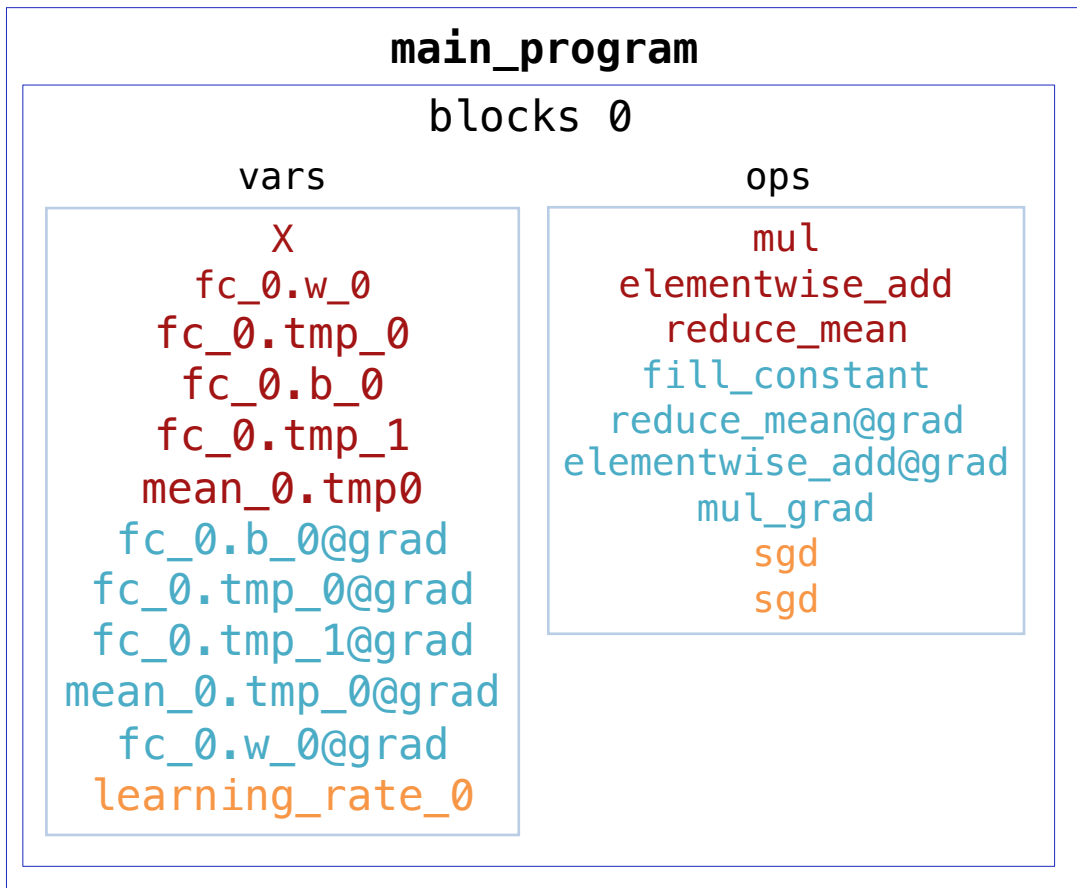
```
optimize_ops = self._create_optimization_pass(params_grads)  
return optimize_ops
```

插入优化OP后返回

— minimize之后的模型



- main_program长这样



`loss = mean(X * W + b)`

Program只是对模型的一个静态描述，在实际运行之前，vars和ops都只是在protobuf中存了一些描述信息，不存在真正的实体

ops按计算图拓扑序排列，通过inputs和outputs成员关联输入输出的vars：

```
message OpDesc {  
  required string type = 3;  
  repeated Var inputs = 1;  
  repeated Var outputs = 2;  
  repeated Attr attrs = 4;  
  optional bool is_target = 5 [ default = false ];  
};
```

— 要真正运行模型，得构造一个执行器

```
exe = static.Executor()
```

```
class Executor(object):  
    def __init__(self, place=None):  
        .....  
        p = core.Place()  
        p.set_place(self.place)  
        self._default_executor = core.Executor(p)  
        .....
```



```
py::class_<platform::Place>(m, "Place")  
    .def(py::init<>())
```

通过pybind绑定了C++端的对应类，
构造C++端Executor的实例对象



```
py::class_<framework::Executor>(m, "Executor")  
    .def(py::init<const platform::Place &>())
```

- C++端的Executor

```
class Executor {
```

C++端的Executor类只有一个Place成员

```
private:
```

```
const platform::Place place_;
```

```
};
```

Place用来表示设备信息，继承自boost::variant，可以理解成一种类型安全的union，是一种多类型单值的数据结构，Place类型相同的数据存储在相同的设备上

```
class Place : public boost::variant<CUDAPlace, XPUPlace, NPUPlace, CPUPlace,  
                                     CUDAPinnedPlace, NPUPinnedPlace> {
```

```
private:
```

```
using PlaceBase = boost::variant<CUDAPlace, XPUPlace, NPUPlace, CPUPlace,  
                                   CUDAPinnedPlace, NPUPinnedPlace>;
```

```
public:
```

```
Place() = default;
```

```
Place(const CPUPlace &cpu_place) : PlaceBase(cpu_place) {} // NOLINT
```

```
Place(const XPUPlace &xpu_place) : PlaceBase(xpu_place) {} // NOLINT
```

```
Place(const NPUPlace &npu_place) : PlaceBase(npu_place) {} // NOLINT
```

```
Place(const CUDAPlace &cuda_place) : PlaceBase(cuda_place) {} // NOLINT
```

```
.....
```

```
};
```

— 通过 `Executor.run` 接口运行模型

`exe.run(static.default_startup_program())` ← 传入 `Program` , 执行 `Executor`

```
compiled_prog = static.CompiledProgram(  
    static.default_main_program())  
loss_data, = exe.run(compiled_prog, ← 传入 CompiledProgram , 执行 PE  
    feed={"X": x},  
    fetch_list=[loss.name])
```

Python端 `Executor.run` 接口 :

根据传入的模型是 `Program` 还是 `CompiledProgram` , 选择 C++ 端的 `Executor` 或 `ParallelExecutor (PE)` 执行模型 ; 通过 `feed` 映射输入数据 , 通过 `ferch` 获取输出结果

实际调用 `Executor._run_impl` :

1. 根据 `feed` 和 `fetch` 做裁剪 , 删除没有必要的 `OP` 和变量 , 并缓存裁剪后的 `Program`
2. 根据传入的是 `Program` 还是 `CompiledProgram` , 分别调用 `_run_program` 或 `_run_parallel`

– Executor.run_program

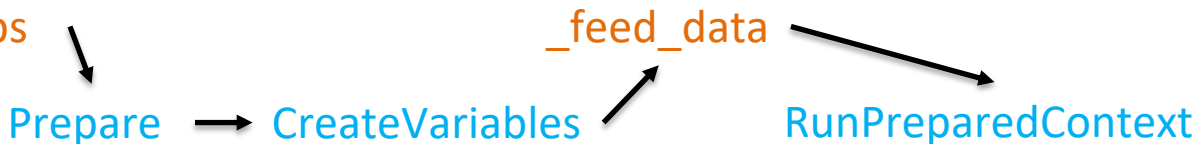
`exe.run(static.default_startup_program())` ← 传入Program，调用_run_program
执行Executor

核心逻辑：

1. 调用_add_feed_fetch_ops在Program中添加feed/fetch OP
2. 调用C++端的Executor.Prepare构造ExecutorPrepareContext
3. 调用C++端的Executor.CreateVariables构造变量实例，放在一个scope中
4. 缓存构造的Program、ExecutorPrepareContext和scope
5. 调用_feed_data将传入的feed数据转换成Tensor，并映射到C++端对应的输入变量中
6. 调用C++端的Executor.RunPreparedContext运行模型
7. 获取执行后输出的fetch变量并返回

Python端：_add_feed_fetch_ops

C++端：



– Executor.Prepare如何构造ExecutorPrepareContext?

```
struct ExecutorPrepareContext {
```

```
.....
```

```
const framework::ProgramDesc& prog_;
```

```
const size_t block_id_;
```

```
std::vector<std::unique_ptr<OperatorBase>> ops_;
```

```
std::unordered_map<const OperatorBase*, std::vector<std::string>>
```

```
unused_vars_;
```

```
bool force_disable_gc_{false};
```

```
};
```

ExecutorPrepareContext中包含一个ops vector存储所有Op实例，和一个unused_vars map记录每个Op执行完之后哪些变量不再需要（用于垃圾回收）

```
for (auto& op_desc : block.AllOps()) {
```

```
    ctx->ops_.push_back(OpRegistry::CreateOp(*op_desc));
```

```
}
```

```
ctx->PrepareUnusedVars(skip_ref_cnt_vars, force_disable_gc);
```

Executor.Prepare依次给每个OpDesc构造真正的Op实例，然后调用ExecutorPrepareContext.PrepareUnusedVars构造unused_vars map

– Executor.CreateVariable如何构造变量实例？

```
for (auto& var : global_block.AllVars()) {  
    auto* ptr = scope->Var(var->Name());  
    InitializeVariable(ptr, var->GetType());  
}
```

→ 首先在scope中新建一个变量

Scope是管理变量的容器，其中包含一个unordered_map，记录每个变量的名字及其对应Variable的unique_ptr，Var接口负责新建一个变量

```
class Scope {  
public:  
    /// Create a variable with given name if it doesn't exist.  
    Variable* Var(const std::string& name);  
  
protected:  
    mutable std::unordered_map<std::string, std::unique_ptr<Variable>,  
        KeyHasher> vars_;  
    .....  
};
```

– Executor.CreateVariable如何构造变量实例？

```
for (auto& var : global_block.AllVars()) {  
    auto* ptr = scope->Var(var->Name());  
    InitializeVariable(ptr, var->GetType());  
}
```

→ 首先在scope中新建一个变量

```
class Variable {  
private:
```

Variable类通过holder_成员持有PlaceholderImpl的shared_ptr

```
    struct Placeholder {
```

```
        void* ptr_;  
        int type_;
```

```
    };
```

PlaceholderImpl是一个模板类，成员obj_表示真正的变量实体，obj_可以有多种类型（可用类型注册在var_type_traits.h中）

```
template <typename T>
```

```
struct PlaceholderImpl : public Placeholder {
```

```
    PlaceholderImpl() { this->Init(&obj_, VarTypeTrait<T>::kId); }
```

```
private:
```

```
    T obj_;
```

```
};
```

```
// pointers to a PlaceholderImpl object indeed.
```

```
std::shared_ptr<Placeholder> holder_;
```

```
};
```

– Executor.CreateVariable如何构造变量实例？

```
for (auto& var : global_block.AllVars()) {  
    auto* ptr = scope->Var(var->Name());  
    InitializeVariable(ptr, var->GetType());  
}
```

首先在scope中新建一个变量
然后对变量做初始化

```
template <typename T>  
T* GetMutable() {  
    if (!holder) {  
        holder_.reset(new PlaceholderImpl<T>());  
    } else {  
        PADDLE_ENFORCE_EQ(  
            holder_->Type(), VarTypeTrait<T>::kId,  
            platform::errors::InvalidArgument(  
                "The Variable type must be %s, but the type it holds is %s.",  
                ToTypeName(VarTypeTrait<T>::kId), ToTypeName(holder_->Type())));  
    }  
    return static_cast<T*>(holder_->Ptr());  
}
```

InitializeVariable通过调用模板函数GetMutable，为Variable构造一个PlaceholderImpl

– Executor.CreateVariable如何构造变量实例？

```
for (auto& var : global_block.AllVars()) {  
    auto* ptr = scope->Var(var->Name());  
    InitializeVariable(ptr, var->GetType());  
}
```

首先在scope中新建一个变量
然后对变量做初始化

PlaceholderImpl的obj_成员可以有多种类型，通常情况下是一个Tensor，Tensor的成员包括内存块holder_，数据类型type_，维度dims_和数据布局layout_等

```
class Tensor {  
private:
```

```
    /*! holds the memory block if allocated. */
```

```
    std::shared_ptr<memory::Allocation> holder_;
```

```
    proto::VarType::type type_;
```

```
    DDim dims_;
```

```
    DataLayout layout_ = DataLayout::kNCHW;
```

```
    size_t offset_;
```

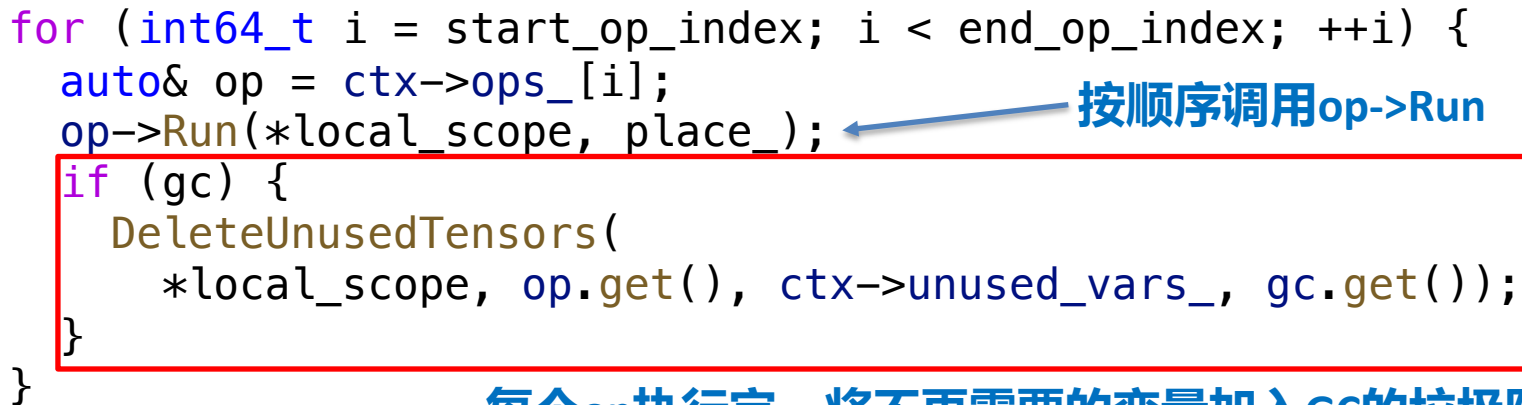
```
    std::shared_ptr<TensorInplaceVersion> inplace_version_counter_;
```

```
};
```

holder_指向底层内存块，在变量实例构造时空，此时并未分配内存

– Executor.RunPreparedContext如何运行模型？

```
for (int64_t i = start_op_index; i < end_op_index; ++i) {  
    auto& op = ctx->ops_[i];  
    op->Run(*local_scope, place_);  
    if (gc) {  
        DeleteUnusedTensors(  
            *local_scope, op.get(), ctx->unused_vars_, gc.get());  
    }  
}
```



按顺序调用op->Run

每个op执行完，将不再需要的变量加入GC的垃圾队列中及时删除

OP执行逻辑：

构造RuntimeContext → ChooseKernel → PrepareData → InferShape → Compute

— 变量内存存在哪里分配？

```
class Tensor {  
private:  
    /*! holds the memory block if allocated. */  
    std::shared_ptr<memory::Allocation> holder_;  
    proto::VarType::type type_;  
  
    DDim dims_;  
    DataLayout layout_ = DataLayout::kNCHW;  
    size_t offset_;  
    std::shared_ptr<TensorInplaceVersion> inplace_version_counter_;  
};
```

holder_ 指向底层内存块，在变量实例构造时空，此时并未分配内存

Tensor在调用mutable_data接口时，才会实际分配内存

模型参数：执行startup_program做初始化时，fill_constant等初始化OP会调用mutable_data

feed输入：NumPy数据转换成Tensor时，SetTensorFromPyArrayT会调用mutable_data

OP运算所需临时变量和输出变量：通常在OP Kernel的Compute函数里调用mutable_data

– mutable_data如何分配内存？

```
void* Tensor::mutable_data(const platform::Place& place,  
    proto::VarType::Type type, size_t requested_size) {  
    type_ = type;
```

```
    size_t size = numel() * SizeOfType(type);
```

先计算一下所需内存空间大小

```
    if (requested_size) {
```

```
        size = requested_size;
```

```
    }
```

```
    if (holder_ == nullptr || !(holder_>place() == place) ||
```

```
        holder_>size() < size + offset_) {
```

```
        holder_.reset();
```

```
        holder_ = memory::AllocShared(place, size);
```

```
        offset_ = 0;
```

然后调用memory::AllocShared
向内存管理模块申请内存

```
    }  
    return reinterpret_cast<void*>(reinterpret_cast<uintptr_t>(  
        holder_>ptr()) + offset_);
```

```
}
```

— 内存管理模块的整体设计

内存管理：本质上是在硬件设备和框架之间实现了一个内存缓冲池

原因：

1. 直接向硬件申请内存的开销较大，设置缓冲池可以减少直接申请内存的次数，从而**提升内存分配和释放的速度**
2. 框架后端支持多种不同的硬件设备，每种硬件设备分配和释放内存的软件接口均不同，内存管理模块可以屏蔽不同硬件的差异，**提供统一的内存分配和释放接口**

总体设计架构可以概括为“两个数据结构 + 一种设计模式”

- **两个数据结构**：Allocator、Allocation
- **一种设计模式**：装饰者模式

— 两个数据结构：Allocator和Allocation

```
class Allocator {  
public:  
    virtual ~Allocator() {}  
  
    using AllocationPtr = std::unique_ptr<Allocation, AllocationDeleter>;  
    inline AllocationPtr Allocate(size_t size) {  
        auto ptr = AllocateImpl(size);  
        ptr->RegisterDecoratedAllocator(this);  
        return AllocationPtr(ptr);  
    }  
  
    inline void Free(Allocation* allocation) {  
        allocation->PopDecoratedAllocator();  
        FreeImpl(allocation);  
    }  
protected:  
    virtual Allocation* AllocateImpl(size_t size) = 0;  
    virtual void FreeImpl(Allocation* allocation);  
};
```

Allocator表示内存分配器，提供Allocate和Free两个方法，用于分配和释放内存

— 两个数据结构：Allocator和Allocation

```
class Allocation {
```

```
.....
```

```
private:
```

```
void* ptr_;  
size_t size_;  
platform::Place place_;
```

Allocation表示内存块，其成员变量包含内存块的指针ptr_，内存块大小size_以及内存所在的设备place_

```
static constexpr size_t kReserveAllocatorNum = 8;  
using DecoratedAllocatorStack =  
    framework::InlinedVector<Allocator*, kReserveAllocatorNum>;  
DecoratedAllocatorStack decorated_allocators_;
```

```
friend class Allocator;  
};
```

- 还有一个decorated_allocators表示该内存块的装饰链
- Allocator采用装饰者模式进行设计，不同的Allocator可以相互装饰，扩展出各种复杂的分配策略，Allocator的装饰顺序就记录在Allocation的装饰链中

— 一种设计模式：装饰者模式

- Allocator的Allocate接口会先调用AllocateImpl分配一块Allocation，然后调用该Allocation的RegisterDecoratedAllocator接口在装饰链中添加自身
- 装饰链中的一个Allocator通常持有下一个Allocator的指针，并在AllocateImpl中调用下一个Allocator的Allocate接口

```
inline AllocationPtr Allocate(size_t size) {  
    auto ptr = AllocateImpl(size);  
    ptr->RegisterDecoratedAllocator(this);  
    return AllocationPtr(ptr);  
}
```



```
inline void RegisterDecoratedAllocator(Allocator* allocator) {  
    decorated_allocators_.emplace_back(allocator);  
}
```

A(1).Allocate() calls -> A(2).Allocate() calls -> ... -> A(n).Allocate() calls



A(1).Allocate() returns <- A(2).Allocate() returns <- ... <- A(n).Allocate() returns

— 一种设计模式：装饰者模式

- Allocator的Free接口会先调用Allocation的PopDecoratedAllocator接口在装饰链中删除自身，然后调用FreeImpl释放内存块
- 在AllocatImpl中通常会调用下一个Allocator的Free接口

```
inline void Free(Allocation* allocation) {  
    allocation->PopDecoratedAllocator();  
    FreeImpl(allocation);  
}
```



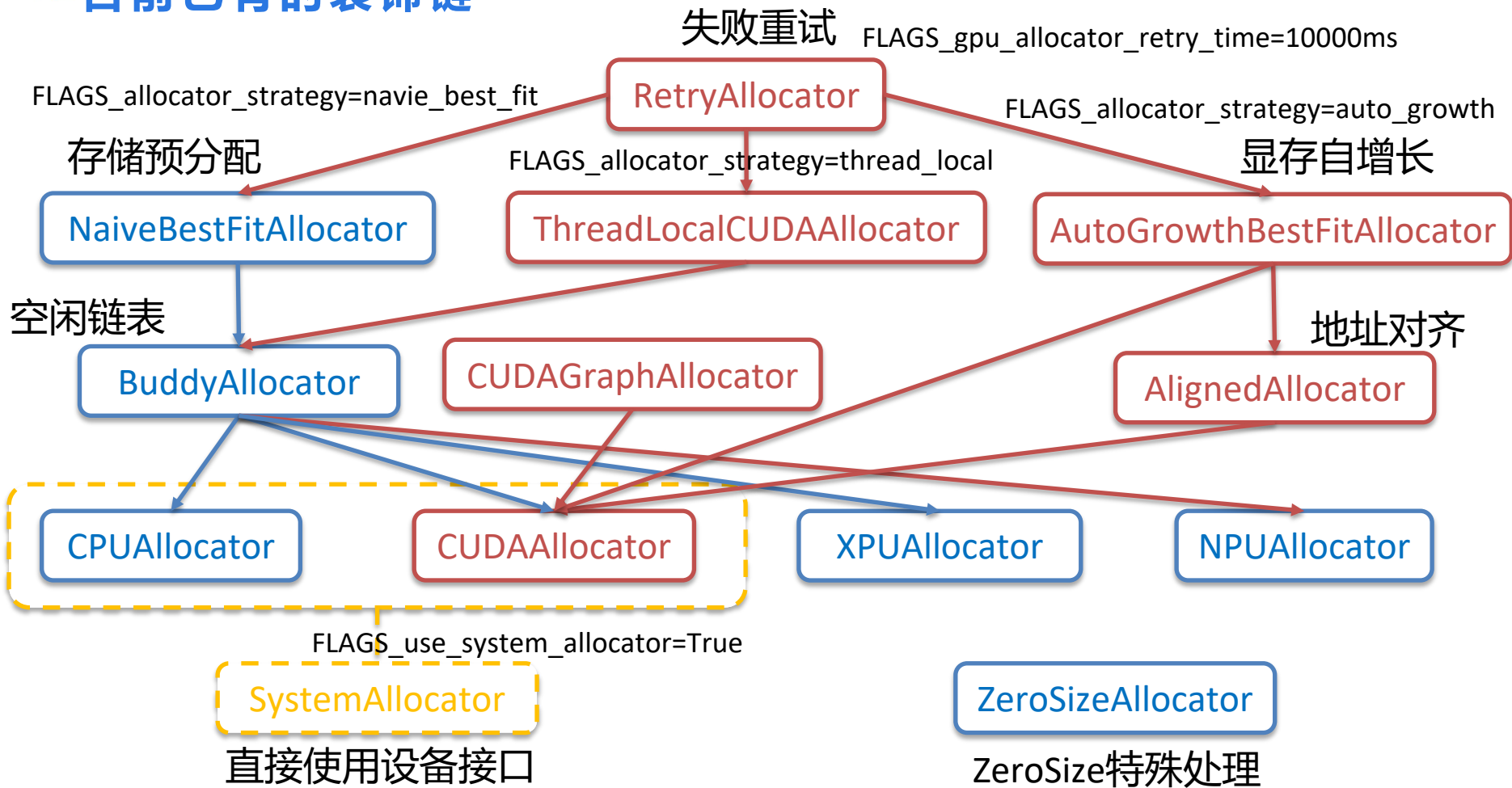
```
inline void PopDecoratedAllocator() {  
    decorated_allocators_.pop_back();  
}
```

A(1).Free() calls -> A(2).Free() calls -> ... -> A(n).Free() calls

A(1).Free() returns <- A(2).Free() returns <- ... <- A(n).Free() returns

□ → □ 只适用于GPU

— 目前已有的装饰链



— 如此多的Allocator，外层如何调用？

通过统一的外观类AllocatorFacade对外提供服务

```
class AllocatorFacade {  
public:  
    ~AllocatorFacade();  
    AllocatorFacade(const AllocatorFacade& o) = delete;  
    const AllocatorFacade& operator=(const AllocatorFacade& o) = delete;  
  
    static AllocatorFacade& Instance();  单例模式
```

```
std::shared_ptr<Allocation> AllocShared(const platform::Place& place,  
    size_t size);
```

```
AllocationPtr Alloc(const platform::Place& place, size_t size);
```

```
private:  
    AllocatorFacade();  
    AllocatorFacadePrivate* m_;
```



对外提供AllocShared和Alloc两个内存申请的接口，
分别返回所申请内存块的share_ptr和unique_ptr

实际的Allocator维护在AllocatorFacadePrivate中

- AllocatorFacadePrivate

```
class AllocatorFacadePrivate {  
public:
```

每个Place有单独的Allocator对象，
通过AllocatorMap记录对应关系



```
    using AllocatorMap =  
        std::map<platform::Place, std::shared_ptr<Allocator>>;
```

```
private:
```

```
    AllocatorMap allocators_;
```

```
    AllocatorStrategy strategy_; → 记录用户配置的内存分配策略
```

```
    static AllocatorMap zero_size_allocators_;
```

```
    static AllocatorMap system_allocators_;
```

```
};
```

```
enum class AllocatorStrategy {
```

```
    kNaiveBestFit,
```

```
    kAutoGrowth,
```

```
    kThreadLocal
```

```
};
```

— 真正的Allocator对象在哪里构造？

```
explicit AllocatorFacadePrivate(bool allow_free_idle_chunk = true) {
    strategy_ = GetAllocatorStrategy();

    switch (strategy_) {
        case AllocatorStrategy::kNaiveBestFit: {.....}
        case AllocatorStrategy::kAutoGrowth: {.....}
        case AllocatorStrategy::kThreadLocal: {.....}
        default: {.....}
    }

    InitZeroSizeAllocators();
    InitSystemAllocators();
    if (FLAGS_gpu_allocator_retry_time > 0) {
        WrapCUDARetryAllocator(FLAGS_gpu_allocator_retry_time);
    }
    CheckAllocThreadSafe();
}
```

AllocatorFacadePrivate的构造函数中，获取用户配置的存储分配策略，为每个设备构造对应的Allocator对象，维护到对应的AllocatorMap中

– AllocatorFacade分配内存的接口

```
std::shared_ptr<Allocation> AllocatorFacade::AllocShared(  
    const platform::Place& place, size_t size) {  
    return std::shared_ptr<Allocation>(Alloc(place, size));  
}
```

```
AllocationPtr AllocatorFacade::Alloc(const platform::Place& place,  
    size_t size) {  
    return m_->GetAllocator(place, size)->Allocate(size);  
}
```



根据place和size在AllocatorFacadePrivate中选择对应的Allocator，然后调用该Allocator的Allocate接口分配内存

— 框架内存申请的接口

```
namespace memory {  
std::shared_ptr<Allocation> AllocShared(const platform::Place  
    &place, size_t size) {  
    return allocation::AllocatorFacade::  
        Instance().AllocShared(place, size);  
}  
  
AllocationPtr Alloc(const platform::Place &place, size_t size) {  
    return allocation::AllocatorFacade::Instance().Alloc(place, size);  
}  
} // namespace memory
```

memory模块进一步包装了AllocatorFacade的单例，对外提供memory::AllocShared和memory::Alloc两个内存申请的接口，分别返回所申请内存块的share_ptr和unique_ptr

— 回来看看mutable_data申请内存的代码

```
void* Tensor::mutable_data(const platform::Place& place,  
    proto::VarType::Type type, size_t requested_size) {  
    type_ = type;
```

```
    size_t size = numel() * SizeOfType(type);
```

```
    if (requested_size) {  
        size = requested_size;
```

```
    }
```

```
    if (holder_ == nullptr || !(holder_>place() == place) ||  
        holder_>size() < size + offset_) {
```

```
        holder_.reset();
```

```
        holder_ = memory::AllocShared(place, size);
```

```
        offset_ = 0;
```

```
    }
```

```
    return reinterpret_cast<void*>(reinterpret_cast<uintptr_t>(  
        holder_>ptr()) + offset_);
```

```
}
```

调用的就是
memory::AllocShared接口

– 为何对外只有Alloc的接口，没有Free？

- Alloc接口返回的智能指针中注册了AllocationDeleter，AllocationDeleter中会获取该Allocation的装饰链最顶端的一个Allocator，然后调用该Allocator的Free接口释放内存
- 内存块的生命周期由C++智能指针进行管理，当智能指针引用计数变为0时，会调用AllocationDeleter释放内存，用户不需要显式进行内存释放的操作

```
class AllocationDeleter {
public:
    inline void operator()(Allocation* allocation) const {
        Allocator* allocator = allocation->TopDecoratedAllocator();
        allocator->Free(allocation);
    }
};

using AllocationPtr = std::unique_ptr<Allocation, AllocationDeleter>;
```

– NaiveBestFitAllocator

- **内存预分配**：第一次分配时直接向系统申请一个超大的内存chunk，chunk可以被切分成很多block，每次分配时从满足size要求的最小block中切分出新块
- NaiveBestFitAllocator实际包装了BuddyAllocator，在Impl函数中通过模板函数Alloc和Free调用了BuddyAllocator的Alloc和Free接口

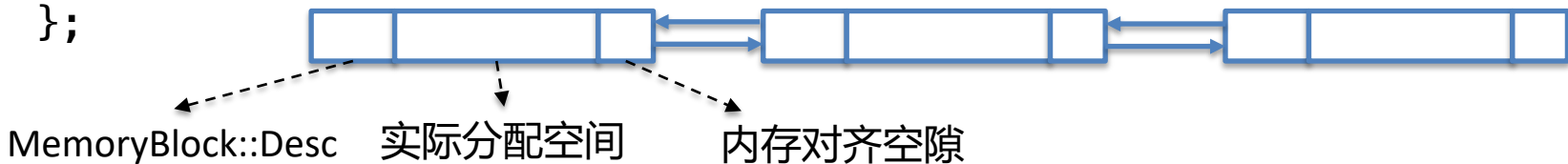
```
template <>
void *Alloc<platform::CUDAPlace>(
    const platform::CUDAPlace &place, size_t size) {
    auto *buddy_allocator = GetGPUBuddyAllocator(place.device);
    auto *ptr = buddy_allocator->Alloc(size);
    .....
}
```

```
template <>
void Free<platform::CUDAPlace>(
    const platform::CUDAPlace &place, void *p, size_t size) {
    GetGPUBuddyAllocator(place.device)->Free(p);
}
```

– BuddyAllocator如何实现？

- 同一个chunk分割出的内存块被组织成一个双向链表
- 用一个MemoryBlock::Desc结构体描述内存块的信息
- CPU内存块描述信息记录在存储空间的前64个字节中

```
struct MemoryBlock {  
    struct Desc {  
        size_t guard_begin = 0;  
        MemoryBlock::Type type = MemoryBlock::INVALID_CHUNK;  
        size_t index = 0;  
        size_t size = 0;  
        size_t total_size = 0;  
        MemoryBlock* left_buddy = nullptr;  
        MemoryBlock* right_buddy = nullptr;  
        size_t guard_end = 0;  
    };  
};
```



– BuddyAllocator如何实现？

```
class BuddyAllocator {
```

```
private:
```

Allocator中内存块用一个index-size-address的三元组表示

```
using IndexSizeAddress = std::tuple<size_t, size_t, void*>;
```

```
using PoolSet = std::set<IndexSizeAddress>;
```

```
size_t total_used_ = 0; // the total size of used memory
```

```
size_t total_free_ = 0; // the total size of free memory
```

```
size_t min_chunk_size_; // the minimum size of each chunk
```

```
size_t max_chunk_size_; // the maximum size of each chunk
```

```
size_t realloc_size_ = 0; // the size of re-allocated chunk
```

```
size_t extra_padding_size_ = 0;
```

```
PoolSet pool_;
```

用一个set<IndexSizeAddress>维护空闲内存池

```
PoolSet chunks_;
```

```
MetadataCache cache_;
```

维护每个内存块指针到Desc信息的map

```
std::unique_ptr<SystemAllocator> system_allocator_;
```

```
std::mutex mutex_;
```

包装了SystemAllocator

```
};
```

— BuddyAllocator内存分配逻辑

在第一次分配时，会调用BuddyAllocator::RefillPool申请一个大chunk填充内存池，RefillPool实现逻辑如下：

1. 计算预分配chunk大小

- $\text{cpu_size} = \min(\text{FLAGS_fraction_of_cpu_memory_to_use} * \text{CPU总内存大小} / 32,$
- $\text{FLAGS_initial_cpu_memory_in_mb})$
- 默认显存大小： $\text{gpu_size} = \text{FLAGS_fraction_of_gpu_memory_to_use} * \text{GPU总内存大小}$
- 可设置预分配显存大小： $\text{gpu_size} = \text{FLAGS_initial_gpu_memory_in_mb}$
- 可设置再分配显存大小： $\text{gpu_size} = \text{FLAGS_reallocate_gpu_memory_in_mb}$

2. 使用SystemAllocator申请一个大内存块chunk

3. 在大内存块的前64个字节中写入描述信息

4. 将该内存块维护到空闲内存池pool_中

- BuddyAllocator内存分配逻辑

```
void* BuddyAllocator::Alloc(size_t unaligned_size) {
```

```
    size_t size = align(unaligned_size + sizeof(MemoryBlock::Desc) +  
        extra_padding_size_, min_chunk_size_);
```

```
    std::lock_guard<std::mutex> lock(mutex_);
```

1. 计算实际占用内存大小，
CPU 4KB对齐，GPU 256B对齐

```
    if (size > max_chunk_size_) {  
        return SystemAlloc(size);  
    }
```

2. 超过最大chunk_size的直接向系统申请

```
    auto it = FindExistChunk(size);
```

3. 在内存池中查找满足size需求的最小空闲块

```
    if (it == pool_.end()) {  
        it = RefillPool(size);  
        if (it == pool_.end()) {  
            return nullptr;  
        }  
    }
```

4. 若没有空闲块满足要求，重新RefillPool

5. 在查找到的空闲块中切割出刚好满足size的
内存块分配出去

```
    total_used_ += size; total_free_ -= size;
```

```
    return reinterpret_cast<MemoryBlock*>(SplitToAlloc(it, size))->Data();
```

```
}
```

- BuddyAllocator内存回收逻辑

```
void BuddyAllocator::Free(void* p) {  
    auto block = static_cast<MemoryBlock*>(p)->Metadata();  
    std::lock_guard<std::mutex> lock(mutex_);  
    auto* desc = cache_.LoadDesc(block);  
    if (desc->get_type() == MemoryBlock::HUGE_CHUNK) {  
        system_allocator_->Free(  
            block, desc->get_total_size(), desc->get_index());  
        cache_.Invalidate(block);  
        return;  
    }  
    block->MarkAsFree(&cache_);  
    total_used_ -= desc->get_total_size();  
    total_free_ += desc->get_total_size();  
    // Trying to merge the right buddy  
    .....  
    // Trying to merge the left buddy  
    .....  
    pool_.insert(IndexSizeAddress(  
        desc->get_index(), desc->get_total_size(), block));  
}
```

1. 超过最大chunk_size的直接释放

2. 标记成空闲块

3. 合并左右空闲块

4. 将回收的内存块放进缓冲池

– AutoGrowthBestFitAllocator如何实现？

- 分割出的内存块在Chunk结构体中表示成一个C++ list
- 用一个Block结构体描述内存块的信息，Block信息直接存储在Chunk结构体中
- 在Allocation中记录指向对应Block的一个iterator

```
struct Chunk {  
    AllocationPtr allocation_;  
    List<Block> blocks_;  
};
```



```
struct Block {  
    void *ptr_;  
    size_t size_;  
    bool is_free_;  
    Chunk *chunk_; // which chunk it is from  
};
```

```
struct BlockAllocation : public Allocation {  
    List<Block>::iterator block_it_;  
};
```

– AutoGrowthBestFitAllocator如何实现？

```
class AutoGrowthBestFitAllocator : public Allocator {
```

```
    using BlockIt = List<Block>::iterator;
```

实际包装了AlignedAllocator，
负责做内存地址对齐

```
    std::shared_ptr<Allocator> underlying_allocator_;
```

```
    std::map<std::pair<size_t, void *>, BlockIt> free_blocks_;
```

```
    std::list<Chunk> chunks_;
```

用一个map维护空闲内存池

```
    size_t alignment_;
```

```
    size_t chunk_size_;
```

```
    bool allow_free_idle_chunk_;
```

不同的chunk用一个list<Chunk>维护

```
    SpinLock spinlock_;
```

```
};
```

— AutoGrowthBestFitAllocator内存分配逻辑

按需分配，若有BestFit的空闲块，则从中分割出所需的block，否则直接向设备申请，实现逻辑如下：

1. 需求的内存大小向256B对齐
2. 在内存池中查找满足size需求的最小空闲块，若找不到则直接向设备申请；若向设备申请失败，会尝试释放一些空闲chunk后再次申请
3. 在查找到或从设备申请到的内存块中切割出满足size要求的最小内存块分配出去
4. 将切割剩下的内存块放回空闲内存池

- AutoGrowthBestFitAllocator内存回收逻辑

```
void AutoGrowthBestFitAllocator::FreeImpl(Allocation *allocation) {  
    std::lock_guard<SpinLock> guard(spinlock_);  
    auto block_it =  
        static_cast<BlockAllocation *>(allocation)->block_it_;  
    auto &blocks = block_it->chunk_->blocks_;  
    block_it->is_free_ = true;
```

1. 标记成空闲块

```
    if (block_it != blocks.begin()) {  
        .....  
    }  
    auto next_it = block_it;  
    ++next_it;  
    if (next_it != blocks.end() && next_it->is_free_) {  
        .....  
    }
```

2. 合并左右空闲块

```
    free_blocks_.emplace(  
        std::make_pair(block_it->size_, block_it->ptr_), block_it);  
    .....  
}
```

3. 将回收的内存块放进缓冲池

– NavieBestFit VS AutoGrowth

整体上都是基于chunk-block的逻辑缓存free-list，使用best-fit原则切割和分配缓存块，但有以下区别：

1. **适用场景**：NaiveBestFit适用于多种设备，AutoGrowth只适用于GPU
2. **申请模式**：NavieBestFit会预分配一个大chunk，AutoGrowth在运行中按需分配

对比项	预分配	按需分配
显存占用	曲线平缓，无论实际使用多少，都会占用大片显存，一张卡上很难同时起多个任务	逐渐递增，用多少申请多少，便于在同一张卡上起多个任务
显存利用率	一个大chunk内存连续，有少量chunk内碎片，基本没有chunk间碎片	多段内存不连续的chunk，既有chunk内碎片，也会产生chunk间碎片
CUDA调用	一开始调用cudaMalloc，后续基本不调用	在训练的前几轮可能会多次调用cudaMalloc
用户使用	需要自己设置预分配显存大小，这很难确定；且无法通过nvidia提供的一些工具观察模型显存增长情况	不需要关心预分配显存大小的问题，且容易观察模型显存增长情况

— 内存管理模块存在的问题

1. 代码略显臃肿

- NaiveBestFitAllocator和AutoGrowthAllocator除了最开始有没有分配一个大chunk，其他缓存和分配逻辑都相同，但这些相同的逻辑在底层有两套完全不同的代码实现
- 向设备直接申请内存的SystemAllocator，在底层也有两套重复的实现

2. 碎片问题仍然严重

3. 不支持多stream

— 常用显存优化方法

随着神经网络的不增大和加深，稀缺的显存资源逐渐成为限制参数规模的瓶颈，需要想方设法减少模型训练时的峰值显存

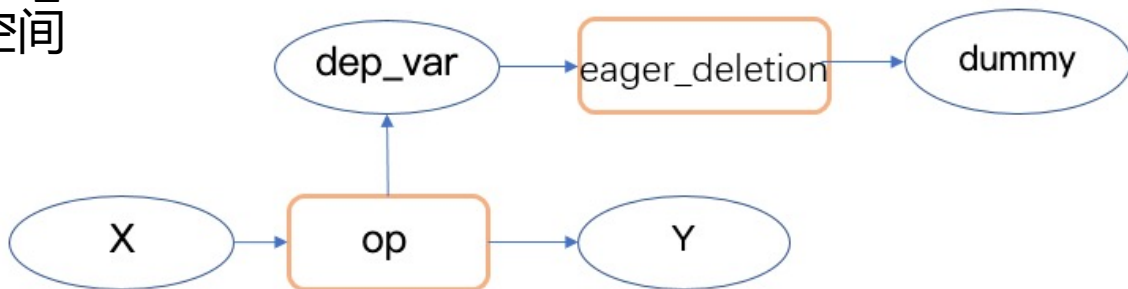
常用显存优化方法：

- 即时垃圾回收
- 显存复用
- swap-out/swap-in
- 模型剪枝
- 分段重计算
- OP融合

— 即时垃圾回收

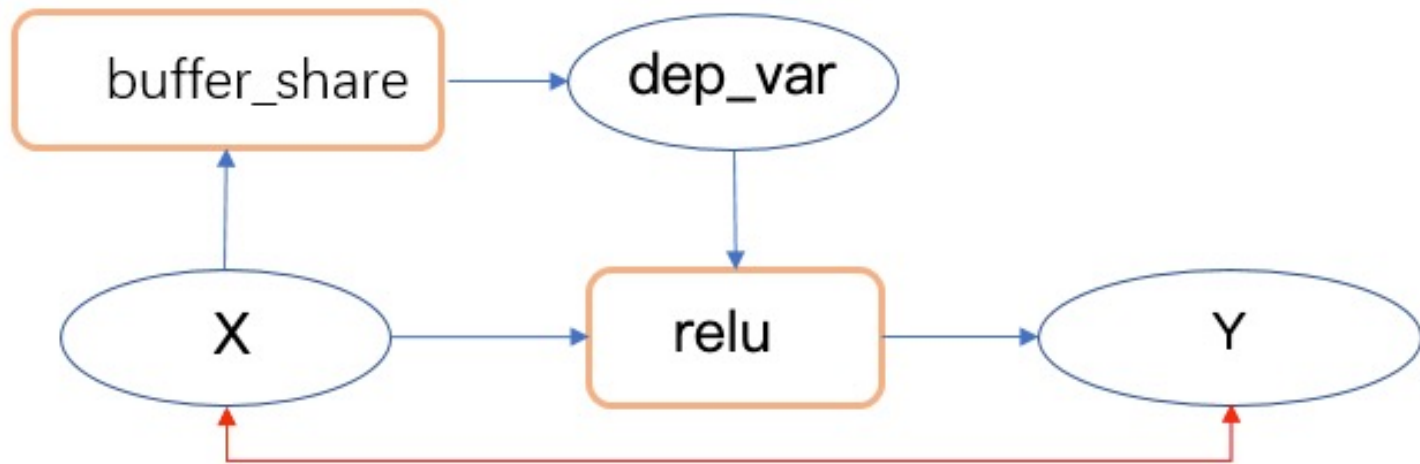
在网络运行阶段及时释放无用变量的存储空间，达到节省显存的目的

- **Executor**：执行前扫描OP列表，记录每个OP运行完后不再需要的变量 `unused_vars`，在该OP执行完后，释放无用变量的内存块
- **PE**：通过 `ReferenceCountPass` 和 `EagerDeletionPass` 实现
 - `ReferenceCountPass`：通过图依赖分析得到每个变量的 `last_live_ops`，用一个引用计数记录 `last_live_ops` 的数量
 - `EagerDeletionPass`：编译时通过 `last_live_ops` 得到可回收变量，在其后插入 `eager_deletion` OP，执行 `eager_deletion` OP 时将对应变量的引用计数减1，当引用计数减为0时释放存储空间



- 显存复用

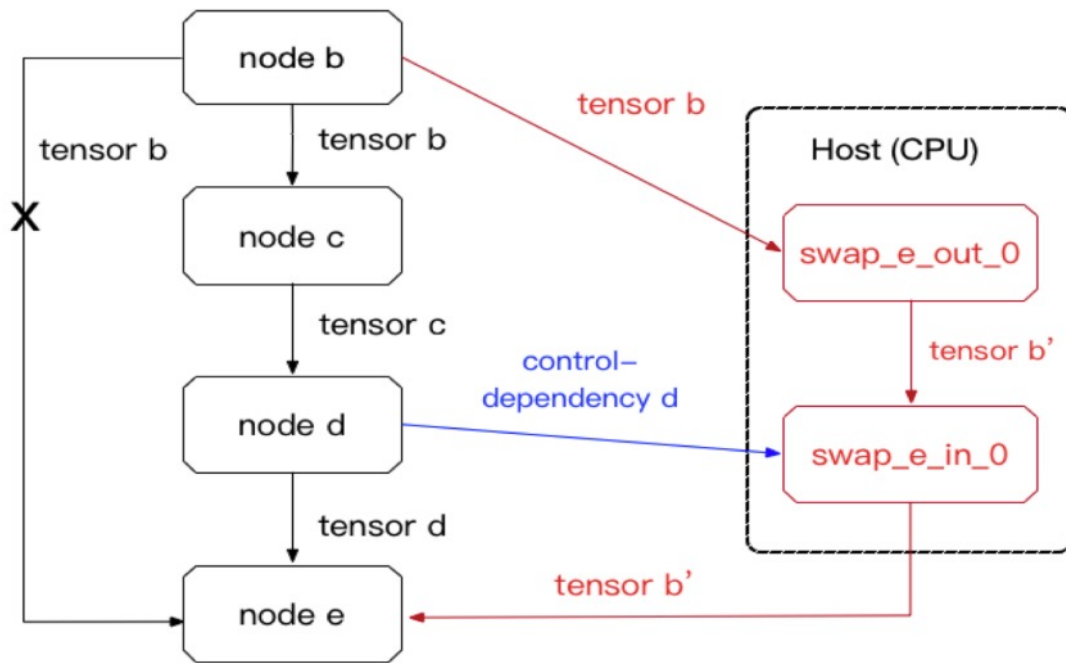
Inplace : OP的输出复用输入的存储空间，通过BufferSharedInplaceOpPass实现



`y.tensor.holder = x.tendor.holder`

– swap-out/swap-in

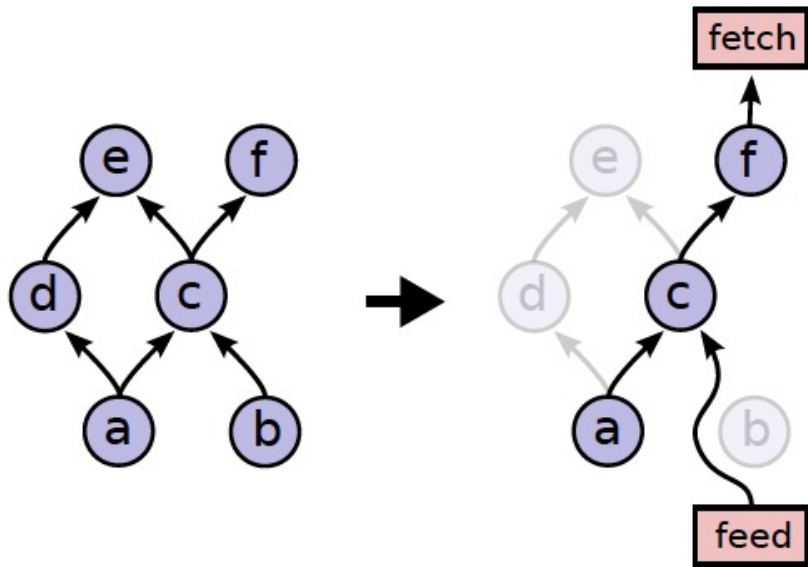
对一些暂时不用的变量swap-out到CPU，需要用到时再swap-in回GPU，起到用时间换空间的效果



— 模型剪枝

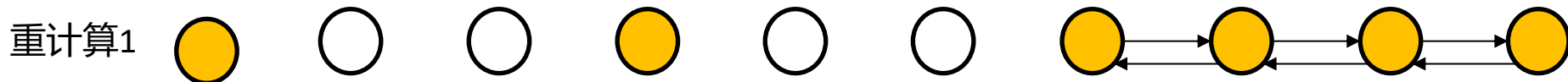
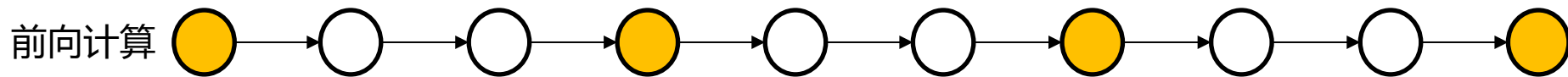
删除不需要计算的OP，裁剪模型

- Optimizer.minimize中no_grad_set参数传入的变量、设置trainable为false的变量、设置stop_gradient为true的变量、以及与loss计算无关的变量，均不需要计算梯度，反向图中与梯度计算和传递无关的部分，可以被裁剪
- 与用户feed/fetch无关的网络部分，可以被裁剪



— 分段重计算

前向计算时只保留一小部分中间结果作为checkpoint，反向时根据保留的中间结果重新计算所需的变量，起到用时间换空间的效果



○ 显存被释放

● 保留作为checkpoint

— OP 融合

通过合并OP计算，减少中间变量的数量

```
t1 = BatchNorm(x)  
out = Relu(t1)
```



```
out = FusedBatchNormRelu(x)
```

— 参考文档

1. PaddlePaddle Fuidl的基本概念：<http://agroup.baidu.com/paddlepaddle/md/article/2721426>
2. Paddle Fuidl框架执行逻辑梳理：<http://agroup.baidu.com/paddlepaddle/view/office/2042688>
3. 静态图自动剪枝：<http://agroup.baidu.com/paddlepaddle/md/article/2574630>
4. PaddlePaddle执行器设计与优化：<http://agroup.baidu.com/paddlepaddle/view/pdf/1853067>
5. 执行器全新架构：<http://agroup.baidu.com/paddlepaddle/md/article/4135742>
6. 内存/显存优化设计：<http://agroup.baidu.com/paddlepaddle/md/article/2721315>
7. Allocator重构：<http://agroup.baidu.com/paddlepaddle/md/article/1609418>
8. Allocator架构设计：<http://agroup.baidu.com/paddlepaddle/md/article/2721317>
9. AutoGrowthAllocator benchmark测试结果：<http://agroup.baidu.com/paddlepaddle/md/article/2721269>
10. Paddle存储管理：<http://agroup.baidu.com/paddlepaddle/view/pdf/2708224>
11. Paddle显存优化串讲：<http://agroup.baidu.com/paddlepaddle/view/office/2042691>
12. 飞桨训练显存分析：<http://agroup.baidu.com/share/md/5086e97124c84094a76e002b9a095500>

THANKS

A horizontal line spanning the width of the slide, positioned below the word 'THANKS'. The line is divided into two equal halves: the left half is red and the right half is blue.

— 串讲现场讨论问题整理答复

框架目前的剪枝只在main_program上做，startup_program是不是也有一些可以剪掉？

@王欢

当startup_program上的一些参数在运行模型时没用到的时候，可以不做初始化操作，直接裁剪掉。但是目前这块没有剪掉，一是可能startup_program需要剪枝的需求场景不多，二是startup_program里需要传入main_program的信息，才能知道哪些参数可以被剪掉，但是用户在运行stratup_program的时候可能还没有main_program的完整信息。后续剪枝这块如果追求极致的性能优化，可以考虑对startup_program也进行裁剪。

— 串讲现场讨论问题整理答复

为什么要有ZeroSizeAllocator？在什么场景下会用到？@红雨

Alloactor有比较长的装饰链，如果要求装饰链上的每个Allocator都去特殊考虑申请ZeroSize的情况，一来实现比较麻烦，二来一个ZeroSize申请要经过很多不必要的Alloactor，会对性能造成影响，所以单独实现一个ZeroSizeAllocator来特殊处理ZeroSize的情况。

使用场景上，主要是对一些空Tensor进行内存申请的操作，可以直接返回一个空指针。之前有一个bug是一些OP操作在计算dims的时候，对空的Tensor会计算出dims是[1]的结果，从而去申请一个size为1的小内存块，有了ZeroSizeAllocator，对于空tensor，就可以直接返回一个空指针，不用再申请内存块。（From嘉彬：这个bug现在已经修复了）

— 串讲现场讨论问题整理答复

NaiveBestFit和AutoGrowth中的空闲池哪个size会更大？@红雨

这个没有实测过，但个人感觉AutoGrowth可能会有更多数量的空闲块，因为按需分配的多个chunk内存地址不连续，容易产生chunk间的碎片。举个例子，考虑一种预分配20MB内存的情况，如果模型先申请两个10MB的内存块，释放后再申请两个6MB的内存块，NaiveBestFit的预分配策略在两个10MB的内存块释放后，再申请的两个6MB的内存块会挨在一起占用12MB的内存，后边剩下1个8MB的空闲块。如果是AutoGrowth的策略，在申请两个10MB的内存块时，会分别申请两个10MB的chunk，在10MB的内存块释放后缓存起来，当再次申请两个6MB的内存块时，会分别从两个chunk里进行切割，这样两个chunk里会各留下一个4MB的空闲块，从而产生2个空闲块。这些空闲块被缓存在空闲池里，会导致空闲池变大，从而后续在查找空闲块时也会变慢。

— 串讲现场讨论问题整理答复

NaiveBestFit中MemoryBlock:Desc如何管理的？GPU和CPU有什么不同？@秋良

NaiveBestFit的BuddyAllocator中维护了一个MetadataCache成员来管理Desc信息，MetadataCache屏蔽了GPU和CPU设备Desc管理的差异。在GPU的场景下，MetadataCache中会维护一个从MemoryBlock地址映射到Desc信息的MetadataMap，通过这个Map来管理每个内存块的Desc信息。在CPU场景下，Desc信息被存储在申请内存块的前64个字节，MetadataCache通过读写内存块的前64个字节来管理内存块的Desc信息。这种把Desc信息存储在分配的内存块上、不用独立数据结构进行管理的方法，可以让内存管理自身的存储开销也被统计进向底层申请的内存中，不会出现实际占用内存超过向底层申请内存大小的情况。

— 串讲现场讨论问题整理答复

一个大Tensor以inplace的方式slice出多个小Tensor，在大Tensor和第一个小Tensor释放的时候，底层的内存块就会被回收。在目前的内存管理模式下，有没有可能让大Tensor释放后的内存块不回收，而是把其持有的内存块的管理权转移给slice出的小Tensor，让每个小Tensor各自独立地管理一小段内存块？@嘉彬

在目前的内存管理模块设计下，内存块的生命周期由C++智能指针进行管理。Allocator会在分配的内存块智能指针中注册一个删除器，该删除器在智能指针析构、引用计数减为0时被调用，从而完成内存块的回收。

要在Tensor析构后将其大内存块的所有权转移给多个小Tensor进行管理，需要实现以下几个功能：

- (1) 在Tensor持有的智能指针析构后，不回收底层内存块。
- (2) 将对一个内存块的管理，转化成对多段小内存块的管理。
- (3) 将多个小内存块的管理权分别移交给slice出的小Tensor。

其中第(3)步属于Tensor之间的操作，比较简单，但第(1)和第(2)步在不修改Allocator的情况下，目前在Tensor这一层均无法实现。

— 串讲现场讨论问题整理答复

一个大Tensor以inplace的方式slice出多个小Tensor，在大Tensor和第一个小Tensor释放的时候，底层的内存块就会被回收。在目前的内存管理模式下，有没有可能让大Tensor释放后的内存块不回收，而是把其持有的内存块的管理权转移给slice出的小Tensor，让每个小Tensor各自独立地管理一小段内存块？@嘉彬

若考虑修改Allocator，第(1)步可以通过修改智能指针中的注册器，让其在释放时不做显存回收来实现，比如可以对需要slice的内存块注册一个空的删除器等。但第(2)步的实现是比较困难的，原因是在Allocator中会维护每个分配出去的内存块的描述信息，这些信息在后续回收和缓存时需要用到，简单的对内存块进行切割和重新管理，难以保证系统中实际存在的内存块与Allocator中描述信息的一致性，所以势必需要同步修改Allocator中维护的内存块描述信息。这个描述信息每种Allocator的维护方式都不一样，有一些甚至是不可修改的。比如NavieBestFitAlloactor的CPU版本，其内存块描述信息就直接存在内存块存储空间的前64个字节中，若将一个内存块slice成多个内存块，就需要多个64字节空间来存储内存块的描述信息，而slice之前的大内存块并不会预留这部分存储空间。对于GPU也会有类似的问题，比如在切割之后如何保证小内存块的内存地址对齐，如果需要再对小内存块做内存对齐操作，又会多出一些内存对齐的空隙，而这些内存对齐空隙的buffer在大内存块分配时也不会预留。

— 串讲现场讨论问题整理答复

一个大Tensor以inplace的方式slice出多个小Tensor，在大Tensor和第一个小Tensor释放的时候，底层的内存块就会被回收。在目前的内存管理模式，有没有可能让大Tensor释放后的内存块不回收，而是把其持有的内存块的管理权转移给slice出的小Tensor，让每个小Tensor各自独立地管理一小段内存块？@嘉彬

综上，在Tensor向内存申请模块申请到一块内存之后，该内存块作为一个整体，其生命周期就只能由Allocator返回的智能指针进行管理，如果Tensor要动Allocator所赋予的智能指针的管理权，这将是一个对Allocator侵入性非常强的操作，最终的实现方案可能会非常tricky，也很难通用化，还会导致Allocator模块的封装性被打破，从而出现一些非常混乱的代码设计。

个人认为，一种更值得思考的实现方案，是把大内存块的切割交还给Allocator来做，通过在Allocator中实现和暴露一个类似SliceAllocation的接口，该接口接受一个内存块的智能指针以及对应的切割要求，返回切割后的多个内存块的智能指针列表。在Allocartor中做内存块的切割和管理权转移，可以比较方便地维护内存块的描述信息，也容易保证新切割的内存块做好内存对齐。但这仍然需要装饰链中的每种Allocator都实现自己的SliceAllocation逻辑，具有较大的工作量。

— 串讲现场讨论问题整理答复

在竞品实现中，Tensor可以传入Allocator，这种设计有什么好处吗？@威行

个人理解，在Tensor中传入Allocator参数，好处可能有两个方面。

一是方便Tensor和Allocator进行交互。拥有了Allocator的信息，在Tensor中就可以很方便地调用Allocator的一些接口对内存块做一些操作，这在实现一些功能时非常有用，比如上一点提到的让Allocator暴露做内存块切割的新接口，Tensor中拥有了Allocator的信息，就可以直接调用该接口。再比如后续Allocator支持多stream后，一些Tensor操作如果涉及stream切换，也需要将信息通过一些接口调用通知Allocator。不过现在在paddle的Allocator装饰者设计模式下，Allocation中本来就拥有装饰链上的所有Allocator信息，所以Tensor要Allocator信息也是可以很方便拿到的。

另一方面，Allocator作为Tensor的参数，也可以很方便地给不同的Tensor绑定不同的Allocator，这样Allocator直接和Tensor进行映射，可以实现一个设备上同时拥有多个Allocator，让Allocator对象的控制粒度更小、更精细。